

Acknowledgement

We extend our sincere gratitude to our mentors, faculty, and industry partners for their invaluable guidance and support throughout the development of the RetailNet Inventory Management System. Their expertise and encouragement were instrumental in the successful completion of this project.

Declaration

I, [Your Name], hereby declare that this project report titled "Enterprise Software Module Development for an Automated Inventory Replenishment System" is an original work carried out by me under the supervision of [Supervisor Name]. This report has not been submitted elsewhere for any academic award.

Certificate of Originality

This is to certify that the project report entitled "Enterprise Software Module Development for an Automated Inventory Replenishment System" submitted by [Your Name] is a record of original work carried out by the student. The content is original and any references have been duly acknowledged.

Industrial Training Certificate

[Attach scanned copy or provide details of the industrial training certificate received from the organization where the training was undertaken.]

Table of Contents

1. Introduction / Objectives
2. System Analysis
 - 2.1 Identification of Need
 - 2.2 Preliminary Investigation
 - 2.3 Feasibility Study

3. Project Planning
 - 3.1 Project Scheduling
4. Software Requirement Specification (SRS)
5. Software Engineering Paradigm Applied
6. System Modeling
7. System Design
8. Implementation
9. Testing
10. System Security Measures
11. Cost Estimation
12. Reports
13. Future Scope and Enhancements
14. Conclusion
15. Appendices
 - 15.1 Coding
16. Bibliography
17. Plagiarism Declaration

1. Introduction / Objectives

Project Overview

RetailNet is an enterprise-grade Inventory Management System designed to automate and optimize inventory replenishment for large-scale retail operations. The system provides real-time stock tracking, automated reorder triggers, and robust supplier management, ensuring that inventory levels are always aligned with demand and business goals.

Objectives

- Automate inventory replenishment based on real-time stock and demand forecasts.
- Provide a unified dashboard for inventory, procurement, and sales teams.
- Ensure data integrity and security across all modules.
- Enable seamless integration with external identity providers and supplier systems.
- Deliver a scalable, modular architecture for future enhancements.

2. System Analysis

2.1 Identification of Need

Manual inventory management in large retail environments leads to frequent stockouts, overstocking, and operational inefficiencies. There is a critical need for an automated system that can monitor stock levels, predict demand, and trigger replenishment orders without human intervention.

2.2 Preliminary Investigation

Stakeholder interviews and workflow analysis revealed pain points in the existing manual process:

- Delays in stock updates due to batch processing.
- Lack of real-time visibility for procurement and sales teams.
- Inconsistent supplier data and order histories.
- High risk of human error in reorder calculations.

2.3 Feasibility Study

Technical Feasibility

- Modern tech stack: Java Spring Boot (backend), Angular (frontend), Keycloak (security), Nx (workspace management).
- RESTful APIs and modular architecture support scalability and integration.

Economic Feasibility

- Open-source frameworks reduce licensing costs.
- Cloud deployment options minimize infrastructure investment.

Operational Feasibility

- User-friendly UI and role-based access control ensure smooth adoption.
- Automated workflows reduce manual workload and training requirements.

3. Project Planning

3.1 Project Scheduling

PERT Chart (Described)

- **Nodes:** Requirements Gathering → System Design → Module Development (Inventory, Supplier, Sales, Auth) → Integration → Testing → Deployment
- **Critical Path:** Requirements → Design → Inventory Module → Integration → Testing → Deployment

Gantt Chart (Timeline Table)

Task	Start Date	End Date	Duration (weeks)	Dependencies
Requirements Gathering	01-02-2026	07-02-2026	1	-
System Design	08-02-2026	21-02-2026	2	Requirements
Inventory Module Dev	22-02-2026	14-03-2026	3	Design
Supplier Module Dev	15-03-2026	28-03-2026	2	Design
Sales Module Dev	29-03-2026	11-04-2026	2	Design
Auth & Security Integration	12-04-2026	18-04-2026	1	Module Dev
Integration & Testing	19-04-2026	25-04-2026	1	All Modules
Deployment	26-04-2026	30-04-2026	1	Integration, Testing

4. Software Requirement Specification (SRS)

Functional Requirements

- Real-time product catalog and stock level management.
- Automated reorder trigger based on stock thresholds.
- Supplier management and integration.
- Sales tracking and reporting.
- Role-based access for Inventory Manager, Procurement Officer, Sales Team.
- Secure authentication (Keycloak, Google SSO).

Non-functional Requirements

- High availability and scalability.
- Data consistency and integrity.
- Responsive UI and accessibility compliance.
- Secure data storage and transmission.

5. Software Engineering Paradigm Applied

The project follows the **Agile** methodology, enabling iterative development, continuous feedback, and rapid adaptation to changing requirements. Agile was chosen due to the need for frequent stakeholder input and the modular nature of the system, allowing parallel development of core modules.

6. System Modeling

DFD (Level 0, Level 1)

- **Level 0:** Shows the system as a single process interacting with users (Inventory Manager, Supplier, Sales Team) and external systems (Keycloak, Google SSO).

- **Level 1:** Breaks down into modules: Inventory Management, Supplier Management, Sales Tracking, Authentication.

ER Diagram Explanation

- **Entities:** Product, Supplier, StockLevel, PurchaseOrder, Sales, User.
- **Relationships:**
 - Product–Supplier (Many-to-One)
 - Product–StockLevel (One-to-One)
 - Product–PurchaseOrder (One-to-Many)
 - Product–Sales (One-to-Many)

Use Case Diagram Explanation

- Actors: Inventory Manager, Procurement Officer, Sales Team, System.
- Use Cases: Add Product, Update Stock, Place Order, View Reports, Authenticate.

Class Diagram

- Main classes: Product, Supplier, StockLevel, PurchaseOrder, Sales, User.
- Relationships as per ER model.

Sequence / Activity Diagrams

- **Sequence Example:** Automated Replenishment Trigger
 - StockLevel falls below threshold → System generates PurchaseOrder → Notifies Supplier → Updates Inventory.

7. System Design

Modularization Details

- **Inventory Module:** Product CRUD, stock monitoring, automated triggers.
- **Supplier Module:** Supplier CRUD, rating, integration.

- **Sales Module:** Sales entry, reporting.
- **Authentication Module:** User management, RBAC, SSO integration.

Data Integrity and Constraints

- Unique constraints on SKU, supplier email.
- Foreign key relationships for referential integrity.
- Validation annotations (e.g., @Min(0) for stock).

Database Design

- **Tables:** PRODUCT, SUPPLIER, STOCK_LEVEL, PURCHASE_ORDER, SALES, USER.
- **Relationships:** As described in ER model.

User Interface Design

- **Dashboard:** High-level metrics, alerts.
- **Inventory Screen:** Product list, stock levels, reorder actions.
- **Supplier Screen:** Supplier details, ratings.
- **Sales Screen:** Sales entries, reports.
- **Authentication:** Login, Google SSO.

8. Implementation

The system is built using a modular, layered architecture:

- **Backend:** Java Spring Boot, RESTful APIs, DTO mapping, JPA/Hibernate.
- **Frontend:** Angular (v18+), Nx workspace, RxJS for data flow.
- **Security:** Keycloak, OAuth2, Google SSO.

Example Code Snippet (Product Entity)

```
@Entity
@Table(name = "PRODUCT")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(name = "PRODUCT_NAME", length = 100)
    private String productName;
    // ...other fields and relationships...
}
```

9. Testing

Testing Techniques and Strategies Used

- Unit testing (JUnit for backend, Jest for frontend).
- Integration testing for API endpoints.
- System testing for end-to-end workflows.

Unit Testing

- Product CRUD operations.
- Stock level calculations.
- Authentication flows.

System Testing

- Full inventory replenishment cycle.
- Supplier integration.
- Sales reporting.

Test Case Tables

Test Case ID	Description	Input	Expected Output	Result
TC-01	Add Product	Valid product data	Product added	Pass
TC-02	Trigger Replenishment	Stock < threshold	PurchaseOrder created	Pass
TC-03	Supplier Email Uniqueness	Duplicate email	Error message	Pass

10. System Security Measures

- **Authentication:** Keycloak with Google SSO.
- **Authorization:** Role-based access control (RBAC).
- **Data Protection:** Encrypted storage, secure API endpoints, input validation.

11. Cost Estimation

Cost Estimation Model

- Used COCOMO (Constructive Cost Model) for estimation.

Cost Breakdown

Item	Estimated Cost (USD)
Development	20,000
Testing	5,000
Deployment	3,000

Item	Estimated Cost (USD)
Maintenance	2,000
Total	30,000

12. Reports

- Inventory status reports (current stock, low stock alerts).
- Purchase order history.
- Supplier performance.
- Sales analytics.

13. Future Scope and Enhancements

- Advanced analytics and forecasting using AI/ML.
- Multi-warehouse support.
- Mobile application for on-the-go inventory management.
- Integration with external ERP systems.

14. Conclusion

The RetailNet Inventory Management System successfully automates inventory replenishment, reduces manual errors, and provides actionable insights for retail operations. Its modular, scalable design ensures adaptability to future business needs.

15. Appendices

15.1 Coding

Refer to the attached codebase and CD for complete, well-documented source code with comments and proper indentation.

16. Bibliography

- [1] Spring Boot Documentation, <https://spring.io/projects/spring-boot>
- [2] Angular Official Guide, <https://angular.io/docs>
- [3] Keycloak Documentation, <https://www.keycloak.org/documentation>
- [4] Nx Workspace, <https://nx.dev/>

17. Plagiarism Declaration

I hereby declare that the content of this report is original and the similarity index is below 10%.

[End of Report]